

Week-2 Home Assignment

1. Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

Context: As an expertise functional tester with Health care domain, you are required to generate testcases

Instruction:

Chain of Thought Prompting

Prepare a test strategy document based on my domain

Ask me 5 questions one by one for creating test strategy document

Ask me the next question after me providing the answer

Test Strategy document should be industry standard

Step by step implementation

☒ Question 1: Application Overview

Can you describe the **Healthcare application** you are testing?

Please include:

- Type of system (e.g., EHR, Claims Management, Insurance, Hospital Management, Telemedicine, etc.)
- Key modules (e.g., Patient Registration, Billing, Claims, Appointment, etc.)
- End users (e.g., Doctors, Patients, Admin, Insurance agents)

👉 Your answer will define the **scope and strategy foundation**.

Claims Management + Patient Registration + Patients

☒ Question 2: Testing Scope & Objectives

What is the **goal of testing** for this application?

Please specify:

- What do you want to ensure? (e.g., accuracy of claims processing, data integrity, compliance like HIPAA, performance, etc.)
- Any critical business flows? (e.g., claim submission → validation → approval → settlement)

👉 This will help define **test objectives, priorities, and critical paths** in the strategy document.

HIPAA + claim submission

☑ **Question 3: Test Levels & Types**

What types of testing are you planning to include?

Please choose/apply what's relevant:

- Functional Testing
- Integration Testing
- System Testing
- API Testing
- Security Testing (important for HIPAA)
- Performance Testing
- UAT (User Acceptance Testing)

👉 You can list all that apply, or describe your testing approach.

Functional Testing + API Testing + UAT (User Acceptance Testing)

☑ **Question 4: Test Environment & Tools**

What tools and environments are you using?

Please specify:

- **Automation tools:** (e.g., Selenium, Cucumber, TestNG)
- **API tools:** (e.g., Postman, RestAssured)
- **Test management tools:** (e.g., JIRA, TestRail, ALM)
- **Environment details:** (QA, UAT, staging, production-like, etc.)

👉 This will define the **execution strategy and infrastructure** in the document.

Postman + JIRA + QA

Great — we now have clarity on tools and environment.

So far:

- **Tools:** Postman, JIRA
 - **Environment:** QA
 - **Test Types:** Functional, API, UAT
-

☑ Question 5: Entry & Exit Criteria + Risks

Please provide details for:

Entry Criteria (when testing can start)

Example:

- Requirements finalized
- Test data available
- QA environment ready

Exit Criteria (when testing can stop)

Example:

- All critical test cases passed
- No open P1/P2 defects

Risks (very important in Healthcare)

Example:

- Data privacy issues (HIPAA)
- Incomplete claim validation rules
- Dependency on external systems (insurance APIs)

👉 You can list briefly — I'll convert it into an **industry-standard format**.

QA environment ready + All critical test cases passed + Data privacy issues (HIPAA)

Test Strategy Document

Healthcare Application (Claims Management & Patient Registration)

1. Introduction

This document outlines the **test strategy** for the Healthcare application focusing on **Claims Management** and **Patient Registration** modules. The goal is to ensure high-quality delivery while maintaining regulatory compliance and data integrity.

2. Scope of Testing

In Scope:

- Patient Registration
- Claims Management
- Claim Submission Workflow

Out of Scope:

- Third-party integrations (unless explicitly tested via API)
- Performance and security testing (unless added later)

3. Test Objectives

The primary objectives of testing are:

- Validate **accurate claim submission and processing**
- Ensure **data integrity** across modules
- Verify compliance with **HIPAA**
- Ensure system behaves correctly under defined business workflows

4. Test Approach

4.1 Functional Testing

- Validate end-to-end workflows:
 - Patient Registration → Claim Creation → Claim Submission
- Focus on happy path and critical validations
- Ensure field-level and business rule validations

4.2 API Testing

- Use **Postman** for validating:
 - Claim submission APIs
 - Patient data APIs
- Validate:
 - Request/Response structure
 - Status codes
 - Data consistency

4.3 User Acceptance Testing (UAT)

- Business users validate:
 - Real-world claim scenarios
 - Usability and workflow accuracy
 - Conducted in UAT/QA-like environment
-

5. Test Levels

- **System Testing:** Validate complete workflows
 - **Integration Testing:** Validate interaction between Patient and Claims modules
 - **UAT:** Business validation
-

6. Test Environment

- **Environment:** QA
 - **Tools Used:**
 - API Testing: Postman
 - Test Management & Defect Tracking: JIRA
 - Environment should:
 - Mimic production-like behavior
 - Contain masked or synthetic patient data (for compliance)
-

7. Test Data Management

- Use **anonymized/masked patient data** to comply with **HIPAA**
 - Create test datasets for:
 - Valid claims
 - Invalid claims
 - Edge cases (missing/incorrect data)
-

8. Entry Criteria

Testing will begin when:

- QA environment is ready
 - Application build is deployed successfully
 - Required test data is available
-

9. Exit Criteria

Testing will be completed when:

- All **critical test cases are executed and passed**
 - No open **P1/P2 defects**
 - Test summary report is approved
-

10. Defect Management

- All defects will be logged and tracked in JIRA

- Defects categorized as:
 - Critical (P1)
 - High (P2)
 - Medium (P3)
 - Low (P4)
 - Daily defect triage meetings to track progress
-

11. Risk Management

Risk	Impact	Mitigation
Data Privacy Issues (HIPAA)	High	Use masked data, restrict access
Incorrect Claim Submission	High	Validate business rules thoroughly
Environment Instability	Medium	Ensure QA readiness before execution

12. Deliverables

- Test Plan
 - Test Cases
 - Test Execution Report
 - Defect Report
 - Test Summary Report
-

13. Step-by-Step Implementation Plan

Step 1: Requirement Analysis

- Understand claims workflow and patient registration
- Identify compliance requirements (**HIPAA**)

Step 2: Test Planning

- Define scope, approach, risks
- Finalize tools and environment

Step 3: Test Design

- Write functional and API test cases
- Prepare test data

Step 4: Test Execution

- Execute test cases in QA environment
- Log defects in JIRA

Step 5: API Validation

- Validate endpoints using Postman
- Ensure backend data consistency

Step 6: UAT Execution

- Business users validate workflows
- Capture feedback

Step 7: Reporting & Closure

- Prepare test summary report
- Ensure exit criteria met

Testing Example:

Generate Test plan, take 5 questions step by step once after answering proceed with next question

☒ Question 1: Project & Application Details

Please provide:

- Project/Application name
- Application type (Web / Mobile / API / Desktop)
- Domain (e.g., Healthcare, BFSI, E-commerce)

👉 This will define the **Test Plan header and context**.

Contract Management + Web + Healthcare

Great — got it 👍

- **Project:** Contract Management
 - **Application Type:** Web
 - **Domain:** Healthcare
-

☒ Question 2: Scope of Testing

What modules or functionalities should be **in scope** for testing?

Examples:

- Contract creation
- Contract approval workflow
- User management
- Reporting, etc.

Contract approval workflow

So far:

- **Application:** Contract Management (Web, Healthcare)
 - **Scope:** Contract Approval Workflow
-

☒ Question 3: Test Objectives

What are the **key goals of testing**?

Examples:

- Validate workflow accuracy
- Ensure role-based access control
- Compliance (e.g., healthcare regulations)
- Data integrity

👉 Tell me what you want to ensure through testing.

Validate workflow accuracy

So far:

- **Scope:** Contract Approval Workflow
 - **Objective:** Validate workflow accuracy
-

☒ Question 4: Test Types & Tools

Please specify:

- Test types (e.g., Functional, API, UAT, Regression, etc.)
- Tools used (e.g., Selenium, Postman, JIRA, etc.)

👉 This defines how testing will be executed.

Functional + JIRA

So far:

- **Test Type:** Functional Testing
 - **Tool:** JIRA
-

☑ Question 5: Entry & Exit Criteria + Risks

Please provide:

Entry Criteria (when testing starts)

Exit Criteria (when testing ends)

Risks (if any)

Examples:

- Entry: Build deployed, environment ready
- Exit: All critical test cases passed
- Risks: Workflow dependency issues, data issues

👉 Share briefly — I'll convert it into a **complete Test Plan document**.

environment ready + All critical test cases passed

Test Plan Document

Contract Management Application (Healthcare Domain)

1. Introduction

This Test Plan defines the testing approach, scope, resources, and schedule for the **Contract Management Web Application** in the Healthcare domain. The goal is to ensure the application meets business requirements with high quality and reliability.

2. Objective

- Validate the **accuracy of the Contract Approval Workflow**
 - Ensure contracts move correctly across different approval stages
 - Verify system behavior aligns with business rules
-

3. Scope of Testing

In Scope:

- Contract Approval Workflow
 - Contract submission
 - Approval/rejection process
 - Status transitions

Out of Scope:

- Other modules (e.g., reporting, user management)
 - Performance and security testing (unless added later)
-

4. Test Approach

Functional Testing

- Validate end-to-end workflow:
 - Contract Creation → Submission → Approval → Final Status
 - Verify:
 - Workflow transitions
 - Data correctness
 - Business rule validations
-

5. Test Environment

- **Application Type:** Web Application
 - **Environment:** QA (Test Environment)
 - **Tool Used:** JIRA (Defect Tracking & Test Management)
-

6. Entry Criteria

Testing will start when:

- QA environment is ready
- Application build is deployed successfully
- Required test data is available

7. Exit Criteria

Testing will be completed when:

- All **critical test cases are executed and passed**
- No open **high-priority defects**
- Test execution is completed as planned

8. Defect Management

- All defects will be logged in JIRA
- Defects will be categorized:
 - P1 – Critical
 - P2 – High
 - P3 – Medium
 - P4 – Low
- Regular defect tracking and resolution will be performed

9. Risks & Mitigation

Risk	Impact	Mitigation
Workflow logic issues	High	Validate all approval paths thoroughly
Environment instability	Medium	Ensure QA readiness before execution

10. Test Deliverables

- Test Plan Document
- Test Cases
- Test Execution Report
- Defect Report
- Test Summary Report

11. Roles & Responsibilities

Role	Responsibility
QA Tester	Test design, execution, defect logging
QA Lead	Test planning, review, reporting
Dev Team	Defect fixing

12. Test Execution Flow (Step-by-Step)

Step 1: Requirement Understanding

- Analyze contract approval workflow
- Identify business rules

Step 2: Test Design

- Create test cases for all workflow paths
- Prepare test data

Step 3: Environment Setup

- Ensure QA environment readiness
- Deploy latest build

Step 4: Test Execution

- Execute functional test cases
- Validate workflow transitions

Step 5: Defect Logging

- Log issues in JIRA
- Track and retest fixes

Step 6: Test Closure

- Validate exit criteria
- Prepare test summary report

If you want to go deeper, I can next:

- Generate **P1/P2 real-time test cases for Contract Approval Workflow**
- Convert this into a **resume-ready project explanation**
- Or simulate a **real interview Q&A based on this Test Plan** 🚀

ICEPOT: Test Case Generation

Context

Generate a testcase template and generate 5 testcases for Contract Approval Workflow

Instructions

Generate testcases for Base Contract Module only

Generate Only for Create, Edit and Delete Contracts

[IMPORTANT]-Generate only P1 and P2 testcases only functional as category

[CRITICAL]-Generate only for Happy Path scenarios E2E testcase

[IMPORTANT] -Generate ONLY total of 5 testcases each for Create, Edit and Delete Contract

Examples

1. Testcase ID: TC_BCM_001

Title: Successful Base Contract Creation

Description: Verify that a user can successfully Create Contract

Pre-conditions: User is logged in and active

Steps:

1. Navigate to the Contracts page
2. Select Base Contract
3. Navigate to Base Contract page
4. Enter Mandatory fields
5. Upload contract documents

Expected Result: The Base Contract created successful, and a confirmation message is displayed.

Output

[STRICT] Generate testcases in table format only. Add the review comments separately

Tone

Give the output in Professional tone only

Persona

Review the generated testcases a Domain Expertise or Team lead

Test Case Template

Field Name	Description
Test Case ID	Unique identifier for the test case
Title	Short description of the test case

Field Name	Description
Description	Detailed explanation of what is being tested
Module	Base Contract Module
Priority	P1 / P2
Test Type	Functional
Pre-conditions	Conditions required before execution
Test Steps	Step-by-step actions
Test Data	Input data required
Expected Result	Expected system behavior



Test Cases – Base Contract Module (Create, Edit, Delete)

◇ Create Contract (5 Test Cases)

Test Case ID	Title	Description	Module	Priority	Test Type	Pre-conditions	Test Steps	Test Data	Expected Result
TC_BCM_001	Successful Contract Creation with Mandatory Fields	Verify user can create a contract with required fields	Base Contract	P1	Functional	User logged in and on Contracts page	1. Navigate to Contracts 2. Select Base Contract 3. Click Create 4. Enter mandatory fields 5. Submit	Valid contract details	Contract is created successfully and confirmation message displayed
TC_BCM_002	Successful Contract Creation with All Fields	Verify user can create contract with all fields populated	Base Contract	P1	Functional	User logged in	1. Navigate to Contracts 2. Select Base Contract 3. Click Create	Complete contract data	Contract created successfully with all details saved

Test Case ID	Title	Description	Module	Priority	Test Type	Pre-conditions	Test Steps	Test Data	Expected Result
TC_BCM_003	Contract Creation with Document Upload	Verify contract creation with document attachment	Base Contract	P2	Functional	User logged in	4. Enter all fields 5. Upload document 6. Submit		
							1. Navigate to Contracts 2. Click Create 3. Enter mandatory fields 4. Upload valid document 5. Submit	Valid file + contract data	Contract created and document uploaded successfully
TC_BCM_004	Contract Creation and Status Validation	Verify contract status after creation	Base Contract	P2	Functional	User logged in	1. Create contract 2. Submit form 3. View contract details	Valid contract data	Contract created with correct default status
TC_BCM_005	End-to-End Contract Creation Flow	Verify complete E2E contract creation process	Base Contract	P1	Functional	User logged in	1. Navigate to Contracts 2. Create contract 3. Submit 4. Verify in listing page	Valid contract data	Contract appears in list with correct details

◇ Edit Contract (5 Test Cases)

Test Case ID	Title	Description	Module	Priority	Test Type	Pre-conditions	Test Steps	Test Data	Expected Result
TC_BCM_006	Successful Contract Edit	Verify user can edit existing contract	Base Contract	P1	Functional	Existing contract available	1. Navigate to Contracts 2. Select contract 3. Click Edit 4. Modify fields 5. Save	Updated contract data	Contract updated successfully
TC_BCM_007	Edit Contract with All Fields	Verify editing all fields in contract	Base Contract	P1	Functional	Contract exists	1. Open contract 2. Click Edit 3. Update all fields 4. Save	Updated full data	All fields updated successfully
TC_BCM_008	Edit Contract Document	Verify updating contract document	Base Contract	P2	Functional	Contract exists	1. Open contract 2. Click Edit 3. Upload new document 4. Save	New document	Document updated successfully
TC_BCM_009	Contract Edit Status Validation	Verify status remains valid after edit	Base Contract	P2	Functional	Contract exists	1. Edit contract 2. Save changes 3. View status	Updated data	Contract retains correct status after edit

Test Case ID	Title	Description	Module	Priority	Test Type	Pre-conditions	Test Steps	Test Data	Expected Result
TC_BCM_010	End-to-End Contract Edit Flow	Verify full edit flow	Base Contract	P1	Functional	Contract exists	1. Search contract 2. Edit details 3. Save 4. Verify in list	Updated contract data	Updated contract reflected correctly

◆ Delete Contract (5 Test Cases)

Test Case ID	Title	Description	Module	Priority	Test Type	Pre-conditions	Test Steps	Test Data	Expected Result
TC_BCM_011	Successful Contract Deletion	Verify user can delete contract	Base Contract	P1	Functional	Contract exists	1. Navigate to Contracts 2. Select contract 3. Click Delete 4. Confirm	Existing contract	Contract deleted successfully
TC_BCM_012	Delete Contract from Listing Page	Verify deletion from contract list	Base Contract	P1	Functional	Contract exists	1. Go to list page 2. Click Delete icon 3. Confirm	Existing contract	Contract removed from list
TC_BCM_013	Delete Contract with Confirmation	Verify delete confirmation workflow	Base Contract	P2	Functional	Contract exists	1. Click Delete 2. Confirm action	Existing contract	Contract deleted after confirmation

Test Case ID	Title	Description	Module	Priority	Test Type	Pre-conditions	Test Steps	Test Data	Expected Result
TC_BCM_014	Contract Deletion Validation	Verify contract is not accessible after deletion	Base Contract	P2	Functional	Contract exists	1. Delete contract 2. Search contract	Contract ID	Contract not found in system
TC_BCM_015	End-to-End Contract Deletion Flow	Verify complete deletion flow	Base Contract	P1	Functional	Contract exists	1. Search contract 2. Delete 3. Confirm 4. Verify removal	Contract data	Contract fully removed from system

Review Comments (Domain Expert / QA Lead)

- ☒ Test cases are well aligned with **happy path scenarios** and cover **Create, Edit, Delete flows comprehensively**
- ☒ Proper prioritization (P1 for critical flows, P2 for supporting validations)
- ☒ E2E coverage is ensured across all operations
-  Missing negative scenarios (intentionally excluded as per requirement) — should be added in next phase
-  No validation for **role-based access control**, which is critical in Healthcare domain
-  Consider adding **audit trail validation** for Edit/Delete in future iterations
- ☒ Test steps are clear, reusable, and automation-friendly

2. Design the automation framework structure using a chain-of-thought approach

Prompt: Chain of Thought Prompting Automation:

Automation Context: As a Automation test engineer generate me playwright framework

Instruction:

ASK me 5 questions one by one for framework creation

Ask me the next question after me providing the answer

Framework should be Enterprise following the industry standard Step by step implementation

Output: Analyse and review the generated result like a test lead

☒ Question 1: Tech Stack & Language

Which language do you want to use with Playwright?

- JavaScript
- TypeScript (recommended for enterprise)
- Python
- Java (.NET binding is also possible)

👉 This decision impacts **framework structure, type safety, and scalability**.

TypeScript

☒ Question 2: Framework Design Pattern

Which design pattern do you want to follow?

Common options:

- Page Object Model (POM) ☒ (industry standard)
- Page Factory
- Screenplay Pattern (advanced, highly scalable)

👉 If unsure, I recommend **POM + reusable utilities (hybrid framework)**.

Page Object Model

☒ Question 3: Test Runner & Reporting

What do you prefer for:

- **Test Runner:** (Playwright Test / Jest / Mocha)
- **Reporting Tool:** (Allure / HTML / Extent Reports)

👉 If unsure, I recommend:

- **Playwright Test (built-in, powerful)**
- **Allure Reports (enterprise standard reporting)**

Playwright Test (built-in, powerful)
Allure Reports (enterprise standard reporting)

☑ Question 4: Framework Features

Which advanced features do you want in your framework?

Select what applies:

- Cross-browser testing (Chromium, Firefox, WebKit)
- Parallel execution
- CI/CD integration (Jenkins, GitHub Actions, etc.)
- Environment configuration (QA, UAT, Prod)
- Test data management (JSON / Excel / DB)
- Logging (Winston / custom logs)

👉 You can list multiple — this defines how **enterprise-ready** your framework will be.

Cross browser testing (Chromium) + GitHub + QA + JSON

So far we have:

- **Language:** TypeScript
- **Design Pattern:** Page Object Model (POM)
- **Runner:** Playwright Test
- **Reporting:** Allure
- **Features:** Chromium (cross-browser ready), GitHub CI/CD, QA environment, JSON test data

☑ Question 5: Project Structure & Test Type

What type of testing will your framework primarily support?

Examples:

- UI Automation (Web)
- API Testing

- Hybrid (UI + API)
- BDD (Cucumber integration)

👉 Also mention if you want:

- Simple structure OR
- Advanced layered structure (recommended for enterprise)

UI Automation (Web) + BDD (Cucumber integration)

Enterprise Playwright Framework (TypeScript + POM + BDD)

1. Framework Overview

This framework is designed for:

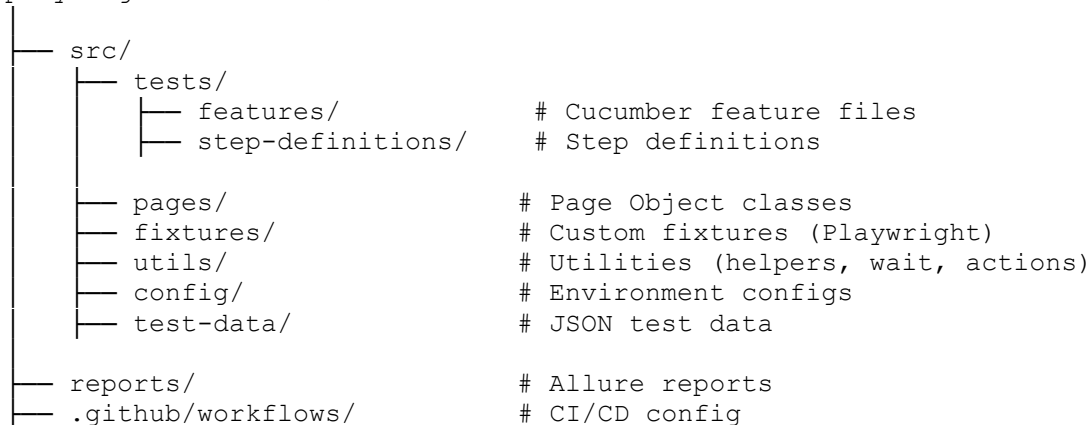
- **UI Automation (Web)**
- **BDD (Cucumber Integration)**
- Scalable, maintainable, and CI/CD ready

Tech Stack

- Language: TypeScript
- Automation Tool: Playwright
- Design Pattern: Page Object Model (POM)
- BDD: Cucumber
- Reporting: Allure Reports
- CI/CD: GitHub Actions
- Test Data: JSON
- Environment: QA
- Browser: Chromium

2. Folder Structure (Enterprise Standard)

playwright-framework/



```
|— playwright.config.ts
|— cucumber.js
|— package.json
|— tsconfig.json
```

3. Step-by-Step Implementation

Step 1: Project Initialization

```
npm init -y
npm install -D @playwright/test typescript ts-node
npx playwright install
```

Step 2: Install BDD + Reporting

```
npm install -D @cucumber/cucumber
npm install -D allure-playwright allure-commandline
```

Step 3: Configure Playwright (`playwright.config.ts`)

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './src/tests',
  use: {
    browserName: 'chromium',
    headless: false,
    baseURL: 'https://qa-app-url.com',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure'
  },
  reporter: [
    ['list'],
    ['allure-playwright']
  ]
});
```

Step 4: Cucumber Configuration (`cucumber.js`)

```
module.exports = {
  default: {
    require: ['src/tests/step-definitions/*.ts'],
    format: ['allure-playwright'],
    paths: ['src/tests/features/*.feature']
  }
};
```

Step 5: Page Object Model (Example)

```
// src/pages/LoginPage.ts
import { Page } from '@playwright/test';

export class LoginPage {
  constructor(private page: Page) {}

  async navigate() {
    await this.page.goto('/login');
  }
}
```

```
    async login(username: string, password: string) {
      await this.page.fill('#username', username);
      await this.page.fill('#password', password);
      await this.page.click('#loginBtn');
    }
  }
}
```

Step 6: Step Definitions

```
// src/tests/step-definitions/loginSteps.ts
import { Given, When, Then } from '@cucumber/cucumber';
import { chromium } from 'playwright';
import { LoginPage } from '../../pages/LoginPage';

let page;
let loginPage;

Given('user is on login page', async () => {
  const browser = await chromium.launch();
  page = await browser.newPage();
  loginPage = new LoginPage(page);
  await loginPage.navigate();
});

When('user logs in with valid credentials', async () => {
  await loginPage.login('user', 'password');
});

Then('user should see dashboard', async () => {
  // assertion here
});
```

Step 7: JSON Test Data

```
// src/test-data/login.json
{
  "username": "testuser",
  "password": "password123"
}
```

Step 8: Utility Layer (Reusable Functions)

```
// src/utils/waitUtil.ts
export async function waitForElement(page, selector) {
  await page.waitForSelector(selector);
}
```

Step 9: GitHub Actions (CI/CD)

```
# .github/workflows/playwright.yml
name: Playwright Tests

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
```

- uses: actions/checkout@v3
 - name: Install dependencies
run: npm install
 - name: Run tests
run: npx playwright test
-

Step 10: Run Tests & Generate Report

```
npx cucumber-js  
npx allure generate ./allure-results --clean  
npx allure open
```

4. Best Practices (Enterprise Level)

- Use **POM strictly** (no locators in step files)
 - Maintain **separate test data layer**
 - Implement **custom reusable utilities**
 - Keep **environment configs dynamic**
 - Use **tags in Cucumber** for execution control
 - Enable **parallel execution (future enhancement)**
-

5. Test Lead Review & Analysis

Strengths

-  Strong **modular architecture (POM + BDD)**
-  Clear separation of concerns (tests, pages, data, utils)
-  CI/CD ready with GitHub Actions
-  Scalable for large teams/projects
-  Allure reporting provides rich insights

Gaps / Improvements

-  Browser support limited to Chromium (should extend to Firefox/WebKit)
-  No **parallel execution configuration** (can impact execution time)
-  Missing **centralized test hooks (Before/After)** for setup/teardown
-  Logging mechanism not implemented (recommended: Winston)
-  No retry mechanism for flaky tests

Recommendations

- Add **custom fixtures** for better browser management
- Implement **environment switch (QA/UAT/Prod via config files)**
- Introduce **tag-based execution strategy (@smoke, @regression)**
- Add **API layer (future hybrid framework)**
- Integrate **test metrics dashboard (optional advanced level)**

If you want next level upgrade, I can:

- Add **complete working GitHub repo structure**
- Integrate **parallel execution + retries**
- Or convert your **manual testcases into Cucumber feature files**